



SOLID принципи и Clean Architecture

Примена SOLID принципа и Clean Architecture на границе и зависности система.

Улога SOLID принципа

Принципи локализују измене, омогућавају замену зависности и олакшавају проверу понашања.

S

Single Responsibility

један кохерентан разлог за промену

O

Open/Closed

проширење без измене стабилног тока

L

Liskov Substitution

заменљивост без нарушавања очекивања

I/D

ISP и DIP

мали уговори и зависност од апстракција

Предуслови за примену SOLID принципа

Уговори, енкапсулација, полиморфизам и одговорност чине основу примене принципа.

ПОЈМОВИ

Interface дефинише очекивано понашање.

Енкапсулација чува инваријанте.

Полиморфизам омогућава заменљиве имплементације.

Апстрактна класа има смисла када постоји стварна заједничка основа.

ПРАКТИЧАН ЕФЕКАТ

код описује намеру

одговорност је видљива

промене су локализоване

тестови лакше прате понашање

Уговори пословног језгра у примеру Logger–Blogger

Доменски слој описује способности система без зависности од складишта и корисничког интерфејса.

```
1 public interface IZapisiServis
2 {
3     Result<ZapisNaSajtu> Objavi(ZapisNaSajtu zapis);
4     Result<IReadOnlyList<ZapisNaSajtu>> PregledZapisa();
5 }
6
7 public interface IBazaPodataka
8 {
9     TabeleBazaPodataka Tabele { get; }
10    bool SacuvajPromene();
11 }
```

НАПОМЕНЕ

Уговор описује стабилну потребу пословног језгра.

Domain не зависи од Database или Presentation пројекта.

Заменљивост долази из јасних уговора, не из назива слојева.

Single Responsibility Principle

Класа може имати више метода када све припадају истој одговорности.

Проблем није број линија кода, већ чињеница да се иста класа мења из више међусобно независних разлога.

НАПОМЕНА

SRP подстиче кохерентне компоненте са јасном одговорношћу.

SRP у сервису Logger–Blogger

Сервис оркестрира објаву преко уговора, без одговорности интерфејса, базе и логовања.

```
1 public sealed class ZapisiServis : IZapisiServis
2 {
3     private readonly IBazaPodataka _baza;
4     private readonly ILogger _logger;
5
6     public Result<ZapisNaSajtu> Objavi(ZapisNaSajtu zapis)
7     {
8         _baza.Tabele.Zapisi.Add(zapis);
9         _baza.SacuvajPromene();
10        _logger.Info("Objavljen zapis");
11        return Result<ZapisNaSajtu>.Ok(zapis);
12    }
13 }
```

НАПОМЕНЕ

Сервис оркестрира сараднике кроз уговоре.

Начин чувања и логовања остаје спољни детаљ.

Ако се промени logger, пословни ток не мора да се мења.

Open/Closed Principle

Апстракције се уводе за познате и оправдане варијације понашања.

Ако стабилан део система стално добија нови switch по типу понашања, променљиво понашање треба сместити иза уговора.

НАПОМЕНА

ОСР омогућава проширење понашања без измене стабилног тока.

ОСР и заменљиво складиште

Нова имплементација складишта додаје се без измене пословног кода.

```
1 public interface IBazaPodataka
2 {
3     TabeleBazaPodataka Tabele { get; }
4     bool SacuvajPromene();
5 }
6
7 public sealed class InMemoryBazaPodataka : IBazaPodataka
8 {
9     public TabeleBazaPodataka Tabele { get; } = new();
10    public bool SacuvajPromene() => true;
11 }
```

НАПОМЕНЕ

Стабилан сервис користи IBazaPodataka.

JSON, in-memory или други provider су технички детаљи.

Проширење се дешава додавањем имплементације.

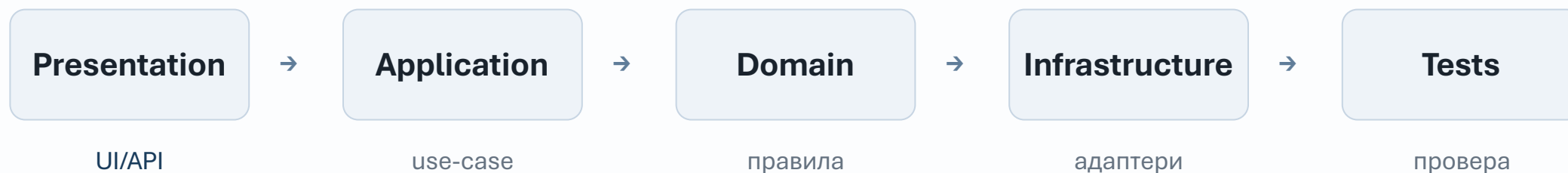
LSP, ISP и DIP у дефинисању граница

Принципи откривају неодговарајуће уговоре и погрешан смер зависности.

Принцип	Проблем	Смер корекције
LSP	подтип мења очекивање	редизајнирати уговор
ISP	клијент зависи од вишка операција	поделити улоге
DIP	use-case креира SQL или HTTP детаљ	зависити од уговора

Смер зависности у Clean Architecture

Називи пројеката могу да се разликују. Смер зависности остаје јасан и проверљив.



НАПОМЕНА

Пословно језгро остаје независно од радног оквира, базе података, HTTP-а и корисничког интерфејса.

Одговорности архитектонских слојева

Квалитет архитектуре процењује се по јасноћи одговорности и смеру зависности.

01

Domain

ентитети, инваријанте и стабилна правила

02

Application

случајеви употребе и портови

03

Infrastructure

база, датотеке и спољни сервиси

04

Presentation

превод захтева у позив use-case-a

Composition root у примеру Logger–Blogger

Једно место бира конкретне имплементације, а пословни код зависи од уговора.

```
1 IBazaPodataka baza = new JsonBazaPodataka("data.json");
2 ILogger logger = new FileLogger("app.log");
3
4 IZapisiServis zapisi = new ZapisiServis(baza, logger);
5 IConsoleIO io = new ConsoleIO();
6
7 var app = new AppContainer(zapisi, io);
8 app.Run();
```

НАПОМЕНЕ

Presentation саставља граф зависности.

Domain и сервиси не креирају JSON базу или FileLogger.

DIP омогућава замену детаља без промене језгра.

Logger–Blogger као студија случаја

Пример служи за анализу одговорности, уговора и зависности.

01

Пословно језгро

правила без
инфраструктурних детаља

02

Adapter-и

конкретна база, датотеке,
логовање

03

Composition root

место где се саставља граф
зависности

Разлози за рефакторисање

Декомпозиција се спроводи када смањује спрегу и локализује измене.

ЛОШ СИГНАЛ

класа мења податке, шаље email и логује

метода има више неповезаних праваца

тест мора да зна приватне детаље

ново понашање тражи измену стабилног центра

ДОБАР СИГНАЛ

једна реченица описује одговорност

сарадници су уговори

промена погађа очекивано место

тестови проверавају понашање

Питања за преглед кода

Питања усмеравају анализу последица дизајнерских одлука.

Питање	Шта открива
Да ли модул има један разлог за промену?	SRP
Да ли нова варијанта мења стабилан ток?	OCP
Да ли имплементација поштује уговор?	LSP
Да ли је интерфејс шири него што клијенту треба?	ISP
Да ли политика зависи од техничког детаља?	DIP

Задатак на вежби

SOLID принципи примењују се на конкретан код и образлажу кроз последице измене.

- 01 Пронаћи класу са најмање две неповезане одговорности.
- 02 Описати њену тренутну улогу једном реченицом.
- 03 Предложити раздвајање и нове границе.
- 04 Образложити које будуће промене постају локализоване.
- 05 Проверити да ли је смер зависности остао исправан.

Критеријум примене

Последица промене треба да остане локализована и предвидљива.

1

једна јасна промена

Када се промени једно пословно правило или технички детаљ, треба да се мења мали и очекиван део система.

Типичне грешке

Механичка примена принципа ствара непотребне апстракције и сложеност.

01

Превише интерфејса

апстракције без реалне варијабилности

02

Погрешно наслеђивање

подтип не поштује уговор

03

Anemic domain

сва правила су у сервисима

04

Framework у језгру

Domain зависи од инфраструктуре

Резиме

SOLID и Clean Architecture штите разумљивост, заменљивост и проверљивост система.

Добар дизајн није онај који има највише слојева, већ онај у коме се одговорност, смер зависности и последица промене могу јасно објаснити.

НАПОМЕНА

Наредна вежба разматра положај пословне логике у доменском и апликационом слоју.

Припрема за наредну вежбу

Наредна вежба разматра положај пословне логике и случајева употребе.

- 01 Изабрати један use-case у свом домену.
- 02 Означити Domain, Application и Infrastructure одговорности.
- 03 Пронаћи једну класу са нејасном одговорношћу.
- 04 Предложити refactoring са образложењем.
- 05 Припремити пример за code review.